

פרויקט תשתית – מערכת איסוף וריכוז אירועים

יחידת Special Data Force

מסמך אפיון והנחיות לצוותי הפיתוח

רקע מבצעי

במהלך פעילות מודיעינית מתמשכת נאספים דיווחי אירועים מאזורים שונים. האירועים מתקבלים ממקורות חיצוניים ומכילים מידע גולמי בלבד: סוג אירוע, אזור, רמת איום, חותמת זמן ותיאור בסיסי.

בשלב זה, היחידה אינה עוסקת בניתוח או קבלת החלטות, אלא בהקמת **תשתית ליבה** שתאפשר איסוף, ריכוז ושימור מידע.

פנייה לצוות

נבחרתם לבנות את שכבת התשתית של מערכת מבצעית.

המשימה שלכם אינה לכתוב "קוד חכם", אלא קוד ברור, יציב וניתן להרחבה ותחזוקה.

כל רכיב במערכת חייב להיות מובן, מבודד, ולפעול בהתאם לאחריות שהוגדרה לו. חריגה מהנחיות אלו תיחשב ככשל תהליכי.

מטרת הפרויקט

בניית מערכת Backend בסיסית מבוססת FastAPI, שתאפשר:

- קליטת אירועים ממקור חיצוני (API של משטרת שיקגו, קישור מופיע למטה בהמשך)
- שמירת האירועים לקובץ JSON
- קריאה מה-JSON והכנסה למסד נתונים MongoDB

4. שליפת אירועים מהמסד לצורכי בדיקה ותצוגה

⚠ דגש קריטי

זהו שלב תשתיתי בלבד.
אין לבצע:

- ולידציה
- עיבוד
- סינון
- שינוי מבנה נתונים

המידע עובר ונשמר **As-Is**.

חלוקת עבודה בצוות (חובה)

הפרויקט מבוצע בזוגות.
לכל חבר צוות תחום אחריות ברור שאינו חופף.

מפתח א' – קליטת אירועים (Ingestion)

אתה אחראי על כניסת המידע למערכת.

תחומי אחריות:

- מימוש endpoint ב-FastAPI
- שליפת אירועים ממערכת חיצונית - **API של משטרת שיקגו הנמצא בקישור הבא (במקרה הזה עם הגבלה ל-20 רשומות):**

[https://data.cityofchicago.org/resource/ijzp-q8t2.json?\\$limit=20](https://data.cityofchicago.org/resource/ijzp-q8t2.json?$limit=20)

- שמירת הנתונים לקובץ JSON

גבולות אחריות:

- ❌ אין חיבור למסד נתונים
- ❌ אין עיבוד, ניקוי או שינוי נתונים

מפתח ב' – ריכוז, שימור ושליפה (Storage & Retrieval)

אתה אחראי על שכבת הנתונים.

תחומי אחריות:

- מימוש endpoint לקליטת נתונים מקובץ JSON והכנסתם למסד הנתונים
- מימוש endpoint נוסף לשליפת נתונים מהמסד (כולם או כמות מוגבלת, לדוגמה 50)

גבולות אחריות:

- אין קריאה למערכת החיצונית ❌
- אין שינוי מבנה הנתונים ❌

מבנה פרויקט מצופה

המערכת תיבנה במבנה בסיסי, פשוט וברור, המאפשר הרחבה עתידית.

```
/project-root
├── /app
│   ├── main.py
│   └── /services
│       ├── external_api_service.py
│       ├── json_service.py
│       └── db_service.py
├── /models
├── /data
│   ├── events.json
│   ├── README.md
│   └── requirements.txt
```

הנחיות מבניות – פנייה אישית לצוות

main.py

זהו קובץ הכניסה של המערכת.

כל ה-endpoints מוגדרים כאן.
הקובץ אינו מכיל לוגיקה מורכבת.

ב-main.py מותר:

- הגדרת endpoints
- קריאה לפונקציות משירותים

אסור:

- כתיבת לוגיקה עסקית
- טיפול ישיר בקבצים, DB או API חיצוני

/services

כאן מתבצעת כל הפעולה האמיתית של המערכת.

כל שירות:

- מטפל באחריות אחת בלבד
- מכיל פונקציות קטנות וברורות
- אינו תלוי ב-FastAPI

דוגמאות לאחריות:

- תקשורת עם API חיצוני
- קריאה וכתיבה ל-JSON
- חיבור ושליפה ממסד נתונים

/models

תיקיה זו קיימת כהכנה להמשך.

בשלב זה אין להשתמש בה ואין להגדיר מודלים או סכמות.

קובץ JSON הוא תוצר ביניים בלבד.

הנתונים נשמרים כפי שהתקבלו, ללא שינוי.

Endpoints נדרשים (ברמה קונספטואלית)

המערכת תכלול לפחות:

- Endpoint לקליטת אירועים ושמירתם ל־JSON
- Endpoint לטעינת אירועים מ־JSON למסד הנתונים
- Endpoint לשליפת אירועים מהמסד (לצורכי בדיקה / תצוגה)

⚠ אין דרישה לפילטרים, ולידציה או ותכולות מתקדמות נוספות.

Git Workflow – סטנדרט עבודה מחייב

העבודה מתבצעת עם Git branching לאורך כל הפרויקט.

מבנה ענפים

```
main
dev — L
*-feature/ingestion — |
*-feature/storage — L
```

כללי עבודה

- אין עבודה ישירה על `main`
- כל פיצ'ר מפותח בענף משלו

- Merge ל-dev דרך Pull Request בלבד
 - Code Review הדדי – חובה
-

Commits – דגש מיוחד

בפרויקט זה:

- עובדים עם הרבה קומיטים קטנים
- כל Commit מייצג שינוי אחד ברור
- הודעות Commit חייבות להיות:
 - מקצועיות
 - קריאות
 - מתארות פעולה אחת

✓ תקין:

Add service for loading events from JSON

✗ לא תקין:

fix, update, test

היסטוריית הקומיטים היא חלק מהותי מהפרויקט.

Code Review – קריטריונים לבדיקה

בעת ביקורת קוד בין חברי הצוות יש לבדוק:

- קריאות הקוד
- שמות פונקציות ומשתנים
- אחריות יחידה לכל פונקציה

- היעדר לוגיקה מורכבת ב-`main.py`

- שימוש נכון בפונקציות עזר משירותים

אישור Pull Request משמעו:
קראתי, הבנתי, ואני עומד מאחורי הקוד.

קריטריונים להצלחה

הפרויקט ייחשב מוצלח אם:

- כל ה-`endpoints` פועלים
- הנתונים נשמרים תחילה ל-JSON
- הנתונים מוכנסים למסד הנתונים
- ניתן לשלוף נתונים מהמסד דרך `endpoint`
- מבנה הקוד ברור ועקבי
- `Git history` משקף תהליך עבודה מקצועי

סיכום

זהו פרויקט תשתיתי ארגוני.
המטרה אינה לכתוב הרבה קוד, אלא:

- לעבוד נכון
- לעבוד בצוות
- לבנות בסיס יציב וברור

שלבי ההמשך יגיעו רק לאחר שהתשתית תעמוד בסטנדרט הנדרש.